

Compiladores

Construção do primeiro tradutor

REALIZAR APÓS posição 11: Aula 09 - Exercícios e Revisão | Gramáticas | Operadores Lógicos | Relacionais | Compiladores

Posições na playlist 1 a 11 | 120 minutos | 100 pontos

Curso principal: Compiladores |

<https://youtube.com/playlist?list=PLX6Nyaq0ebfhI396WIWN6WIBm-tp7vDtV&si=LoaU9IzLMuSVikgi>

Conteúdo verificável: Ambiente; fases; tradução sintática; árvores; descida; FIRST; tradutor C++; lexer; símbolos; escopo; revisão

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Fases e diagnóstico

Para `'x = a + 12; '`, identifique tokens e descreva percurso por análise sintática e tradução. Classifique erro de caractere inválido, falta de operando e nome não declarado. Indique uso de compilador, debugger e ferramenta de build no desenvolvimento do tradutor.

Desenvolvimento / justificativa / esquema:

Derivação e ambiguidade

Gramática $E \rightarrow E + E \mid E * E \mid id$. Dê duas árvores/parentizações para $id + id * id$, explique ambiguidade e proponha gramática que dê precedência a $*$ sobre $+$. Traduza a expressão para pós-fixa sob essa precedência.

Desenvolvimento / justificativa / esquema:

Descida recursiva

Para $E \rightarrow T R$, $R \rightarrow + T R | \epsilon$, $T \rightarrow \text{num}$, determine FIRST e escreva pseudocódigo de parser/tradutor para pós-fixa. Execute em 2+3+4. Indique quando consumir token e como detectar entrada inválida.

Desenvolvimento / justificativa / esquema:

Escopo e símbolos

Programa declara x externo=1; abre bloco, declara x interno=2, usa x; fecha bloco e usa x.

Descreva tabela de símbolos com pilha de escopos, mostre resolução e implemente pseudocódigo de declarar/buscar. Distinga sombrear de declarar duas vezes no mesmo escopo.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Fases e diagnóstico (15 pontos)

Tokens $\text{id}(x)$, $=$, $\text{id}(a)$, $+$, $\text{num}(12)$, $;$. Lexer identifica lexemas; parser reconhece atribuição/expressão; tradução produz representação/saída; tabela de símbolos resolve nomes. Caractere não reconhecido é léxico; operando ausente sintático; nome não declarado refere-se à informação de símbolos/contexto. `g++` compila C++; `gdb` inspeciona execução; `make/cmake` organizam construção/dependências. Não presumir que a linguagem traduzida é C++ só porque tradutor foi implementado nele.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Derivação e ambiguidade (25 pontos)

Parentizações $(\text{id}+\text{id})*\text{id}$ e $\text{id}+(\text{id}*\text{id})$, árvores distintas para mesma cadeia. Gramática $E \rightarrow E+T|T$; $T \rightarrow T*F|F$; $F \rightarrow \text{id}|(E)$ dá multiplicação mais forte. Pós-fixa $\text{id id id } * +$. Essa gramática ainda tem recursão esquerda e não é diretamente adequada à descida recursiva; transformação própria é necessária, sem confundir remover ambiguidade com tornar LL.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Descida recursiva (25 pontos)

$\text{FIRST}(E)=\text{FIRST}(T)=\{\text{num}\}$, $\text{FIRST}(R)=\{+, \epsilon\}$. Pseudocódigo:

```
E(): T(); R()
T(): exigir num; emitir seu valor; avançar
R():
  se lookahead == '+':
    avançar; T(); emitir '+'; R()
  senão se lookahead está em FOLLOW(R): retornar
  senão: erro
```

Em expressão isolada FOLLOW contém fim; saída `2 3 + 4 +`. Exigir fim após E evita aceitar prefixo seguido de lixo. Emitir operador depois de ambos operandos produz pós-fixa correta.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Escopo e símbolos (35 pontos)

Uso interno resolve2, externo1. Pilha de mapas: push ao abrir e pop ao fechar. declarar verifica apenas mapa corrente para duplicata; buscar percorre do topo até raiz. Sombrear em escopo novo pode ser permitido pela linguagem; duplicata no mesmo escopo pode ser erro.

```
declarar(nome,info):  
    se nome em topo: erro  
    topo[nome] = info  
buscar(nome):  
    para escopo do topo à raiz:  
        se nome em escopo: retornar escopo[nome]  
    erro de nome não declarado
```

Tempo de busca depende profundidade/estrutura; não cobrar análise semântica de tipos ainda não introduzida.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.